

Ejercicio 4: Alquiler de Botes en el Río Ebro (Programación Dinámica)

Enunciado

Sobre el río Ebro hay n estafetas de correos. En cada estafeta se puede alquilar un bote que permite ir a cualquier otra estafeta río abajo (es imposible remontar la corriente). La tarifa indica el coste del viaje de i a j para cualquier punto de partida i y cualquier punto de llegada j más abajo en el río.

Puede suceder que un viaje de i a j sea más caro que una sucesión de viajes más cortos, en cuyo caso se tomaría un primer bote hasta una estafeta k y un segundo bote para continuar a partir de k . No hay coste adicional por cambiar de bote. Diseñar un algoritmo eficiente que determine el coste mínimo para ir de i a j .

1. Definición del Estado

Definimos una matriz bidimensional DP de tamaño $n \times n$ donde:

- $DP[i][j]$: El **coste mínimo** de viajar desde la estafeta i hasta la estafeta j (con $i \leq j$), pudiendo realizar paradas intermedias de transbordo.

2. Relación de Recurrencia

Para ir de la estafeta i a la j , podemos optar por realizar el viaje directo o bien realizar una parada intermedia en alguna estafeta k ($i < k < j$) para continuar desde allí con otro bote:

$$DP[i][j] = \min_{i \leq k < j} \{ DP[i][k] + T[k][j] \}$$

Donde $T[k][j]$ representa el coste directo de la tarifa de alquiler de bote entre las estafetas k y j .

3. Casos Base

- **Estar en la misma estafeta:** El coste de ir de una estafeta a sí misma es nulo porque no nos desplazamos.

$$DP[i][i] = 0 \quad \forall i \in [1, n]$$

4. Algoritmo en Pseudocódigo

Calculamos las soluciones de abajo hacia arriba de forma iterativa utilizando un bucle de diferencia de índices ($gap = j - i$) para asegurar que las subestructuras ya estén resueltas al evaluar intervalos mayores.

```

Algoritmo CosteMinimoBotes(T, n)
    // Entrada:
    //   - T[1..n][1..n]: Matriz de tarifas de botes directas (T[i][j] para i < j)
    //   - n: Número total de estafetas
    // Salida:
    //   - DP[1..n][1..n]: Matriz de costes mínimos de viaje entre cualquier par

    Crear matriz DP[1..n][1..n]

    // 1. Caso Base: Permanecer en la misma estafeta tiene coste 0
    Para i ← 1 hasta n hacer
        DP[i][i] ← 0
    FinPara

    // 2. Rellenar la tabla por diagonales (según la distancia de viaje 'gap')
    Para gap ← 1 hasta n - 1 hacer
        Para i ← 1 hasta n - gap hacer
            j ← i + gap

            // Inicializar con la tarifa del viaje directo
            DP[i][j] ← T[i][j]

            // Buscar si es más barato hacer una parada intermedia en k
            Para k ← i + 1 hasta j - 1 hacer
                DP[i][j] ← Mínimo(DP[i][j], DP[i][k] + T[k][j])
            FinPara
        FinPara
    FinPara

    Retornar DP
FinAlgoritmo

```

5. Análisis de Complejidad

- **Complejidad Temporal:** $\mathcal{O}(n^3)$. El algoritmo se compone de tres bucles anidados: el primero controla el *gap* (hasta n), el segundo el origen i (hasta n), y el tercero busca de forma lineal el punto intermedio de transbordo k entre i y j .
- **Complejidad Espacial:** $\mathcal{O}(n^2)$ para almacenar la matriz DP de dimensiones $n \times n$.